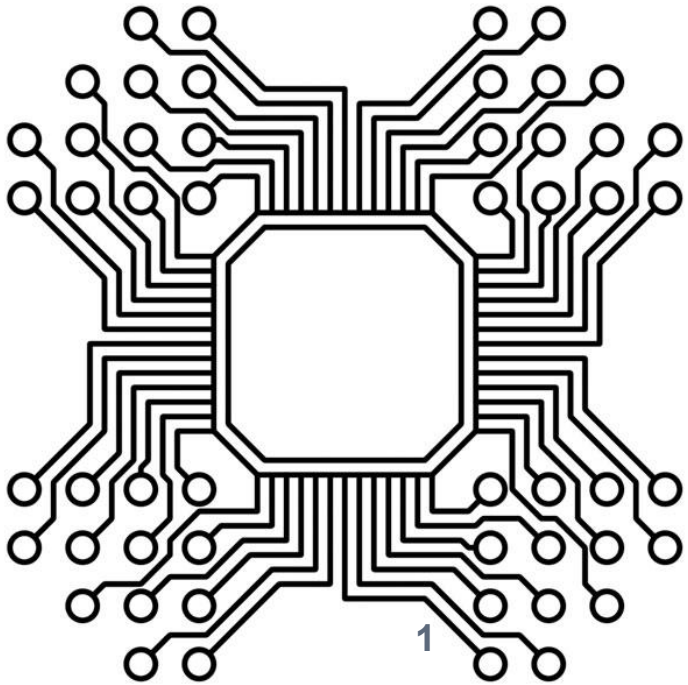
CONNECTING IOT DEVICES

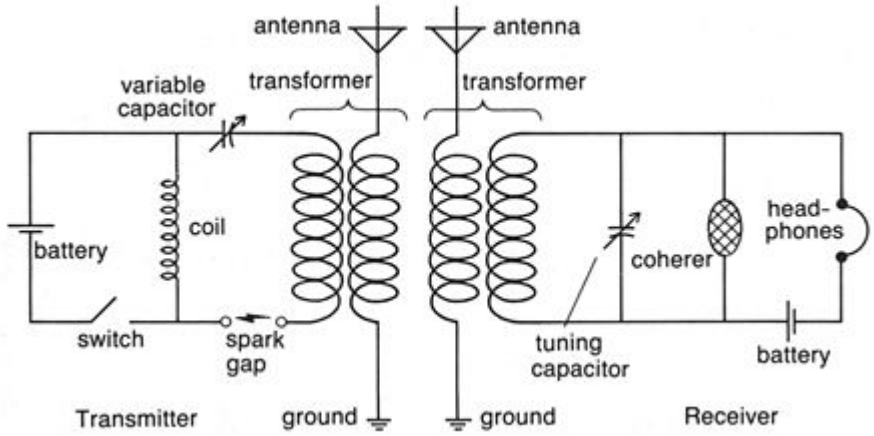
Lecturer: Dr. Bui Ha Duc
Dept. of Mechatronics
Email: ducbh@hcmute.edu.vn



First Wireless communication

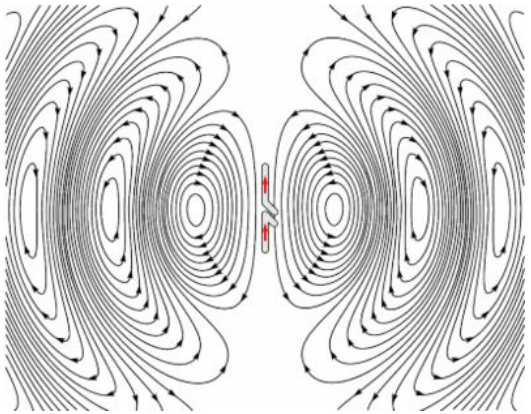


Guglielmo Marconi (1874-1937)



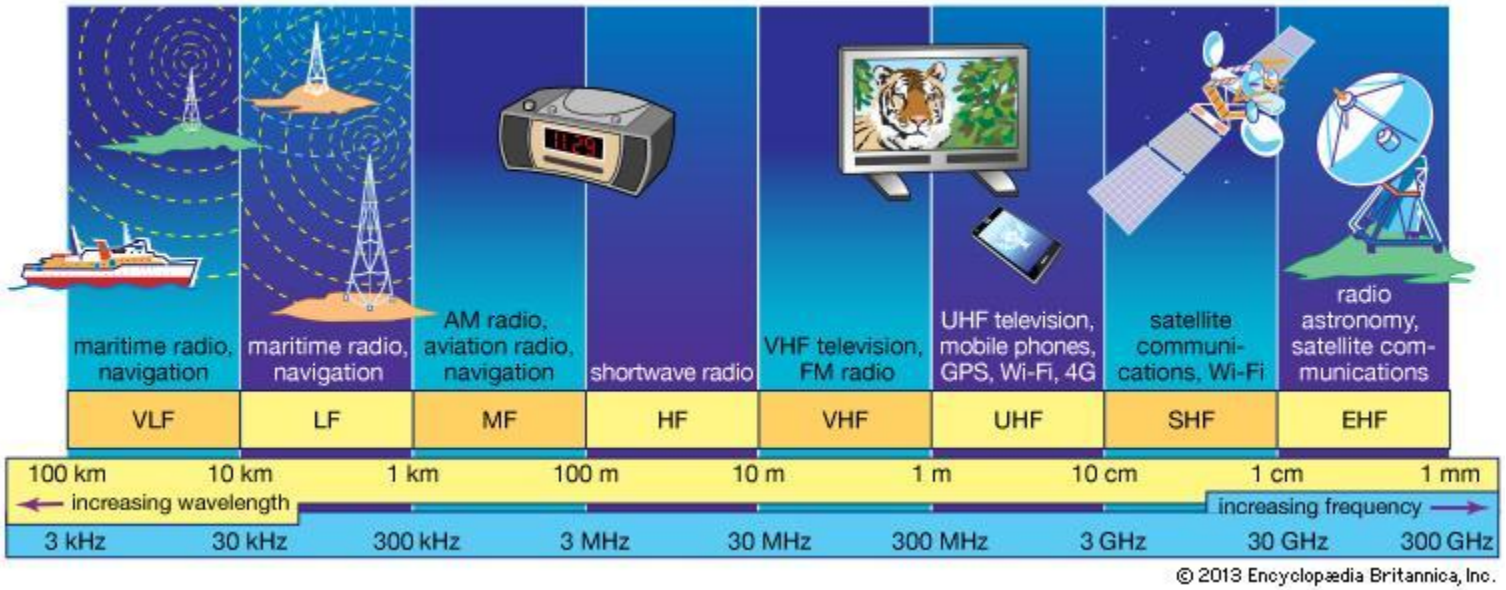
Marconi's 1901 wireless system

http://people.seas.harvard.edu/~jones/cscie129/nu_lectures/lecture6/marconi/marconi.html



RF - Radio frequency

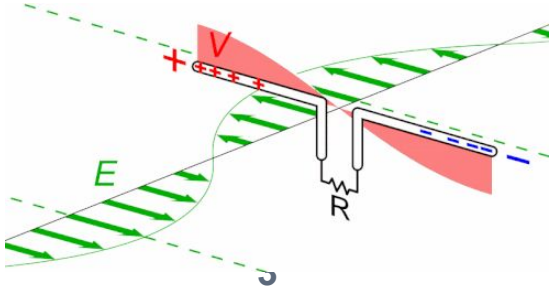
- RF: frequency from few KHz to 300GHz



- Relationship between frequency f and wavelength λ

$$C = f \times \lambda$$

C is the speed of light

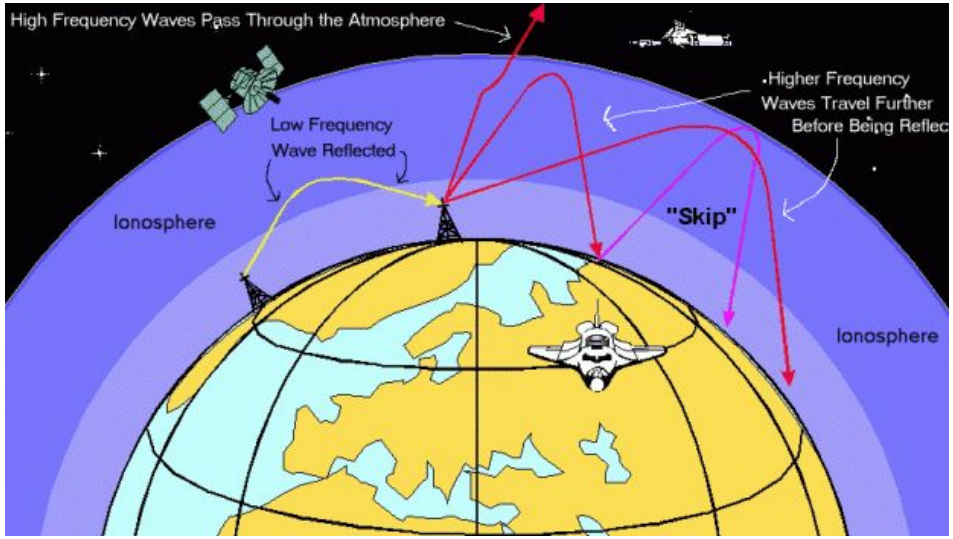


Diffraction: Radio Wave Diffraction vs Wavelength

How does antenna work: How does an Antenna work?

RF - Radio frequency

- Different frequencies of RF waves have different propagation characteristic in the earth's atmosphere:
  - Shortwave RF can reflect of the ionosphere and travel around the world
  - Longwave RF may cover a part of the Earth very consistently



https://www.bugoutbagbuilder.com/blog/what-radio-propagation-rf-propagation

ISM band – 2.4GHz

- ISM: Industry, Science and Medical
- ISM band: **2.4GHz – 2.483 GHz**
- getting more crowded day by day

Why use this band?

- Free
- Avoid Radio, TV, mobile phone band
- Long range
- Small antenna
- Cost
- Microwave frequency 2.5GHz

Sub-GHz band (<1GHz)

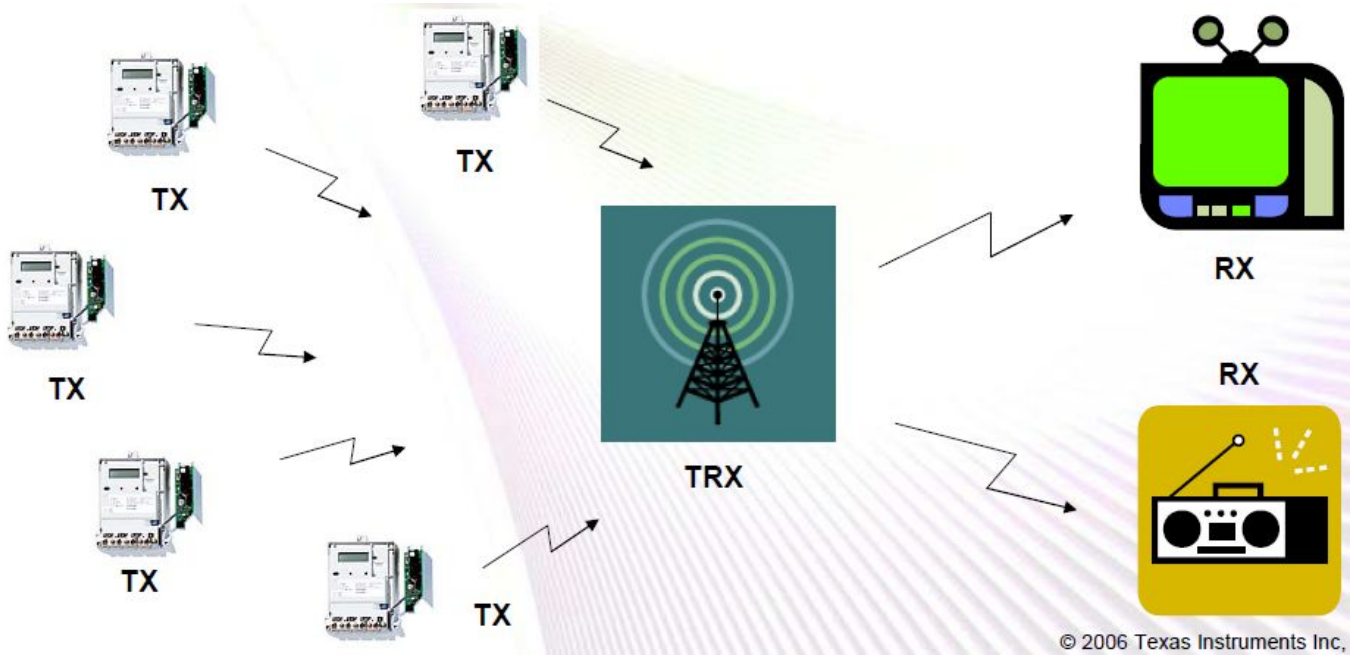
- licensed free
- Frequency less than 1 GHz
  - North America and Australia 915 MHz,
  - Europe 868 MHz,
  - China 470 MHz, and 779 MHz,
  - Japan 426 MHz and 920 MHz.
  - The rest 433MHz
- Mostly use for low power, long range network
  - hundreds of meters indoors, kilometers outdoors.
  - Low data speed (~50kbit/s)

RF Communication System

Simplex RF system

- A radio technology that allows only **one-way communication** from a transmitter to a receiver
- Examples: FM radio, Pagers, TV, One-way AMR systems

Analog or Digital?



RF Communication Systems

Half-duplex RF Systems

- Operation mode of a radio communication system in which each end can transmit and receive, but **not simultaneously**.
- Note: The communication is bidirectional over the same frequency, but unidirectional for the duration of a message. The devices need to be transceivers.





RF Communication Systems

Full-duplex RF Systems

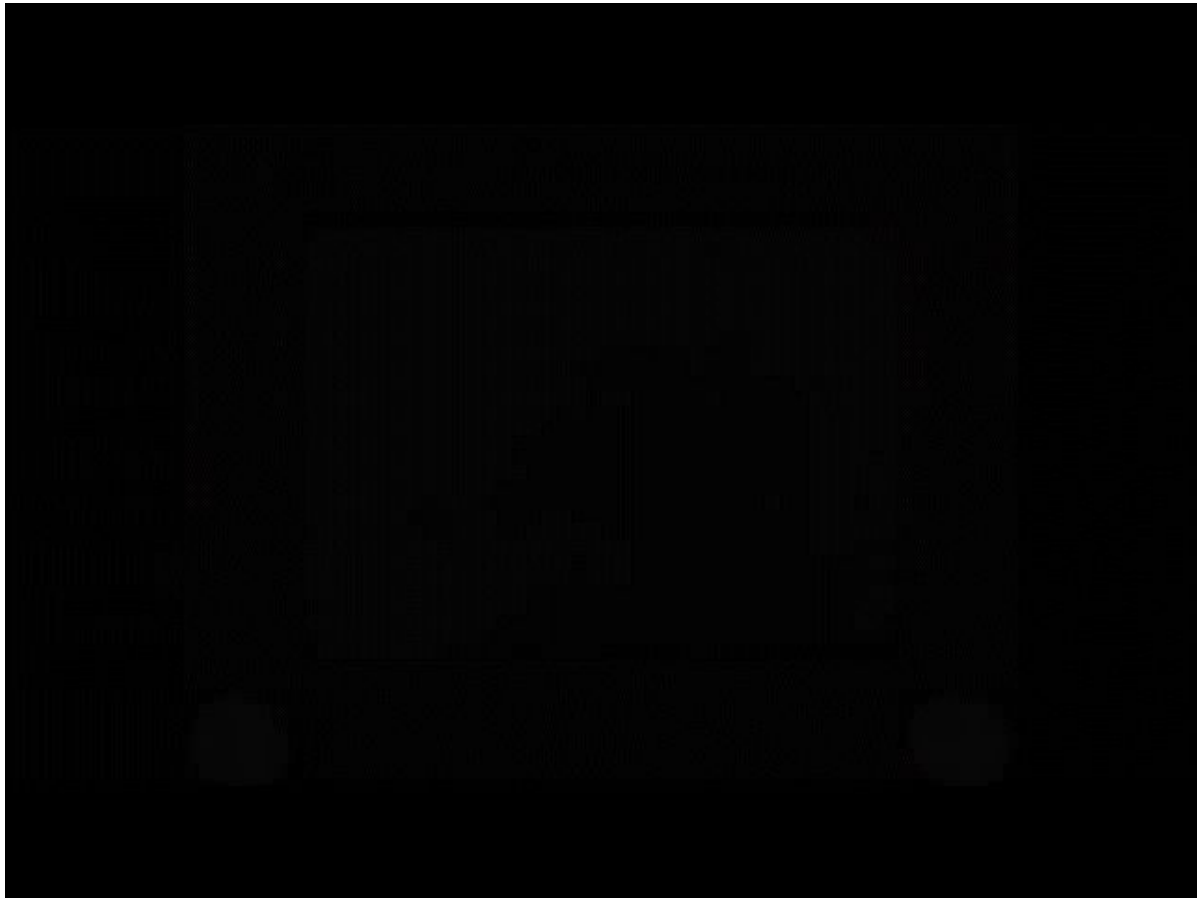
- Radio systems in which each end can **transmit and receive simultaneously**
- Typically **two frequencies** are used to set up the communication channel. Each frequency is used solely for either transmitting or receiving.
- Examples: Cellular phones, satellite communication





How Information Travels Wirelessly

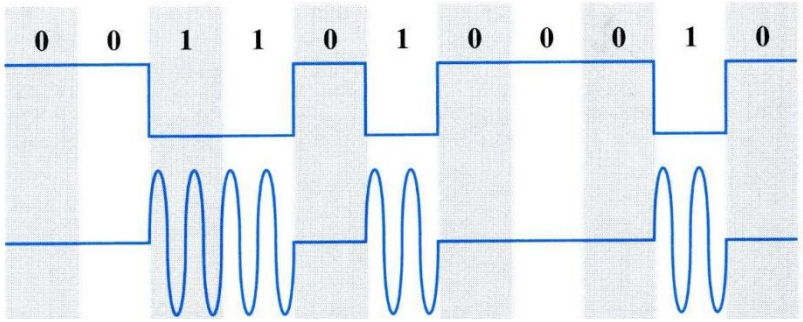
- MIT short introduction



https://www.youtube.com/watch?v=Ax7dYaRiY6o

Digital Modulation

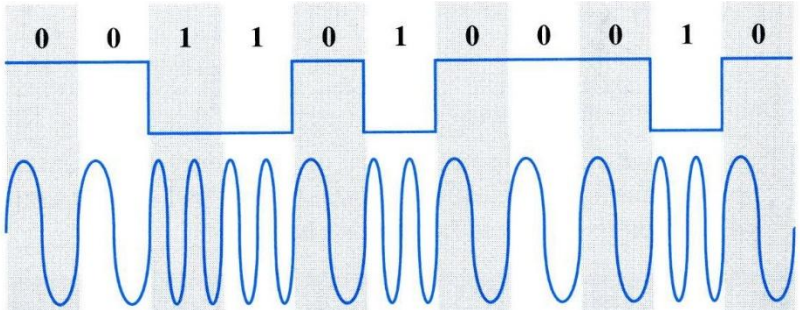
Amplitude Shift Keying (ASK)



Pros: simple

Cons: susceptible to noise

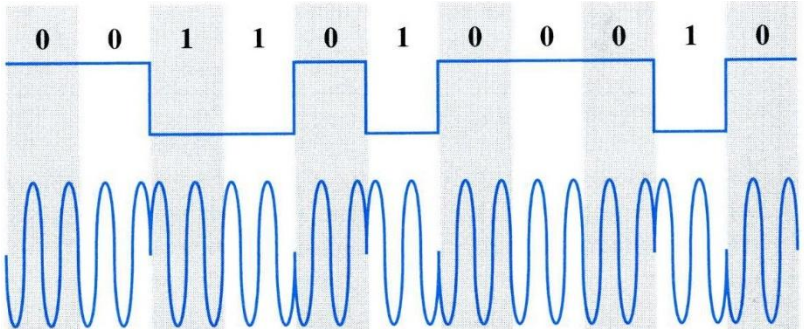
Frequency Shift Keying (FSK)



Pros: less susceptible to noise

Cons: requires larger bandwidth/bit than ASK

Phase Shift Keying (PSK)

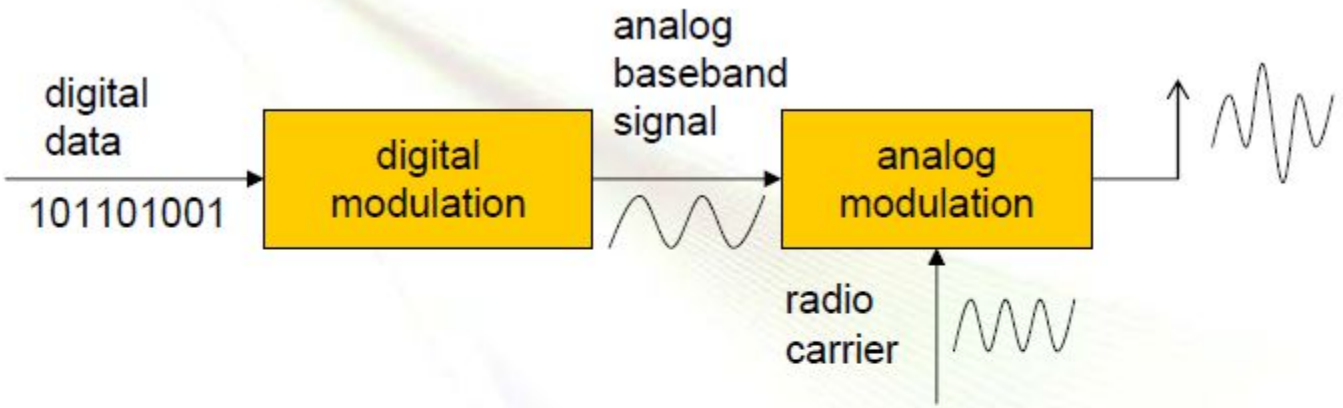


Pros: Less susceptible to noise, Bandwidth efficient

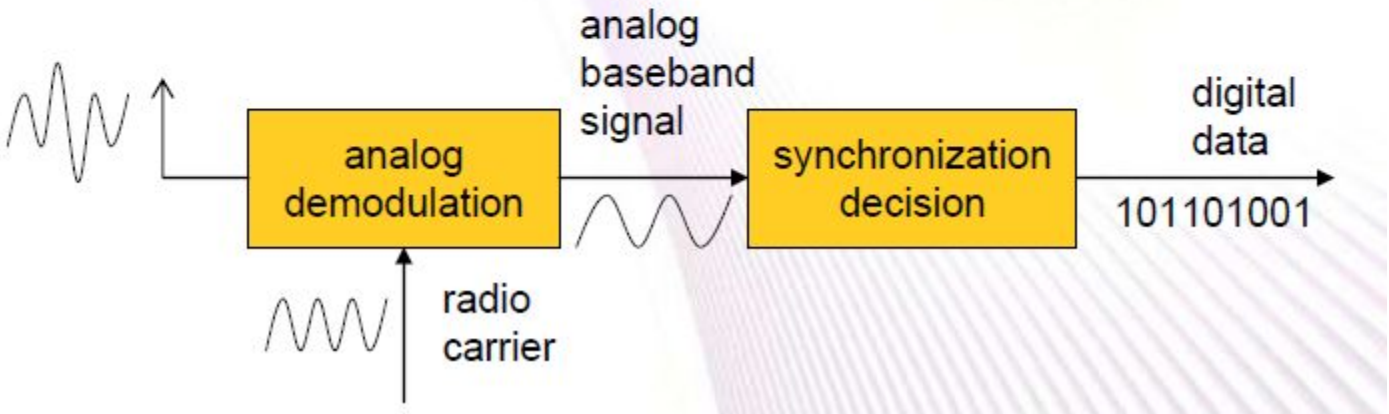
Cons: Require synchronization in frequency and phase

http://ironbark.xtelco.com.au/subjects/DC/lectures/7/

Wireless Communication Systems



Radio Transmitter

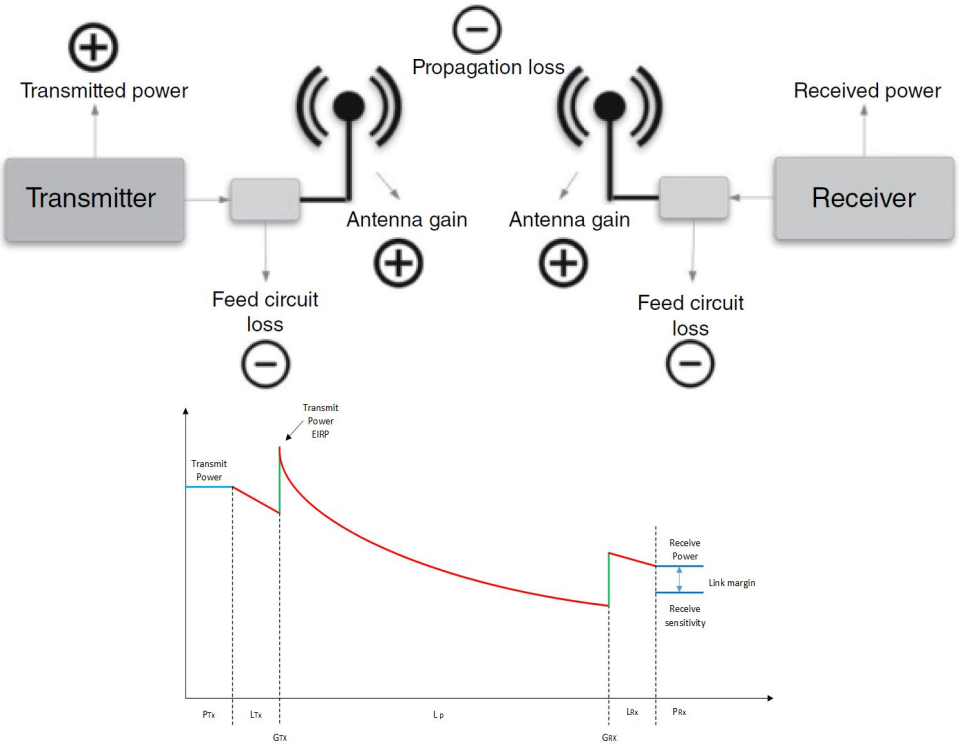


Radio Receiver

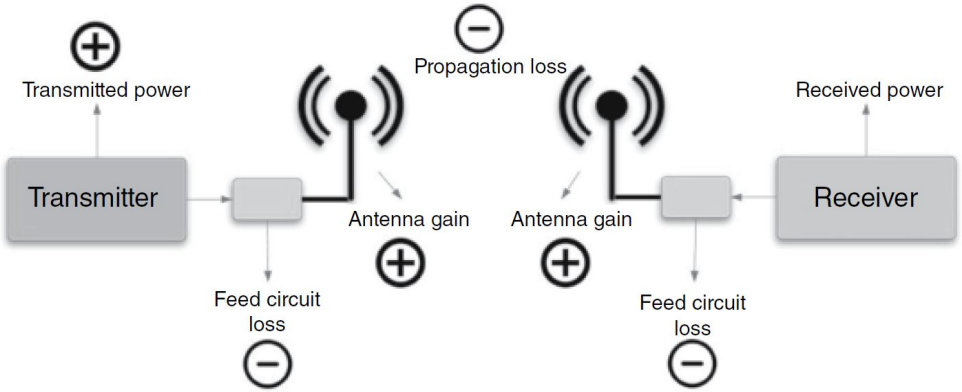
Link budget

- Link budget is a way of quantifying the connection performance

Received power (dBm) = Transmitted power (dBm) + gains (dB) - losses (dB)



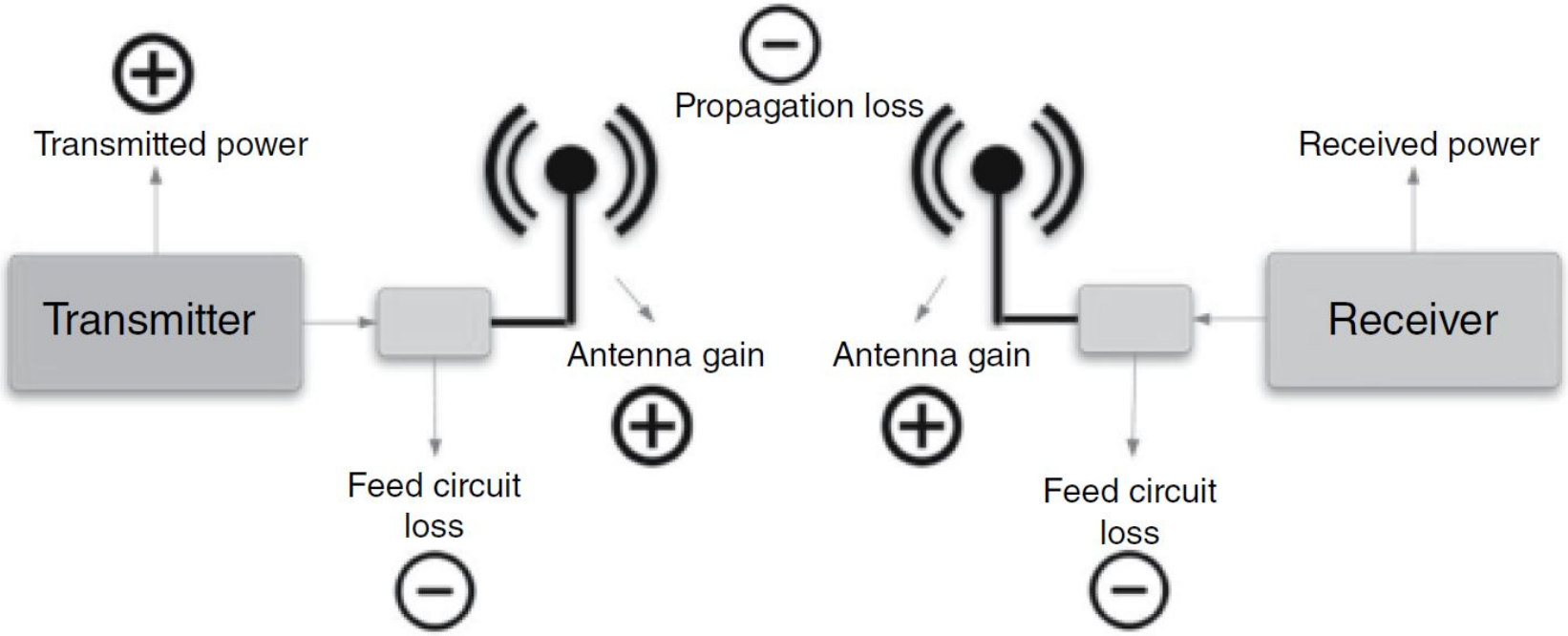
Link budget



$P_{Rx}(dBm) = P_{TX}\ (dBm) + G_{TX}\ (dBi) + G_{RX}\ (dBi) - L_{FS}\ (dB) - L_{TX}\ (dB) - L_{RX}\ (dB)$

where L_{FS}: Free-space Loss (propagation loss)

$$\mathrm{FSPL(dB)} = 20\log_{10}(d_{km}) + 20\log_{10}(f_{GHz}) + 92.45$$



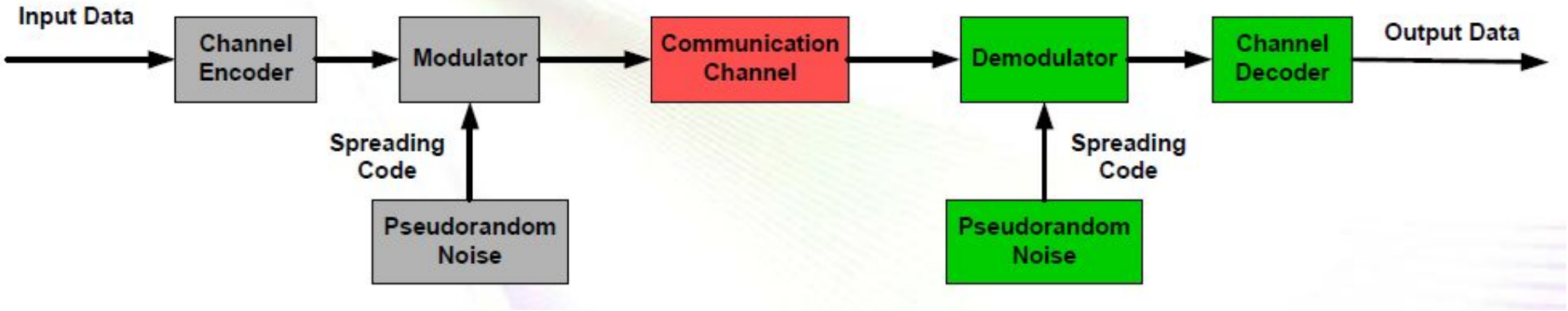
Example A 2.4 GHz antenna of an access point has a gain of 10 dBi, a transmitting power of 20 dBm (equivalent to 100 mW), and a receiving sensitivity of −89 dBm. Five kilometers away, there's a stationary IoT device equipped with a 2.4 GHz antenna of 14 dBi gain, a transmitting power of 30 mW (15 dBm), and a receiving sensitivity of −82 dBm. The cables and connectors have a loss of 2 dB at each end. Is the communication link feasible?

Avoiding Interference in ISM Band

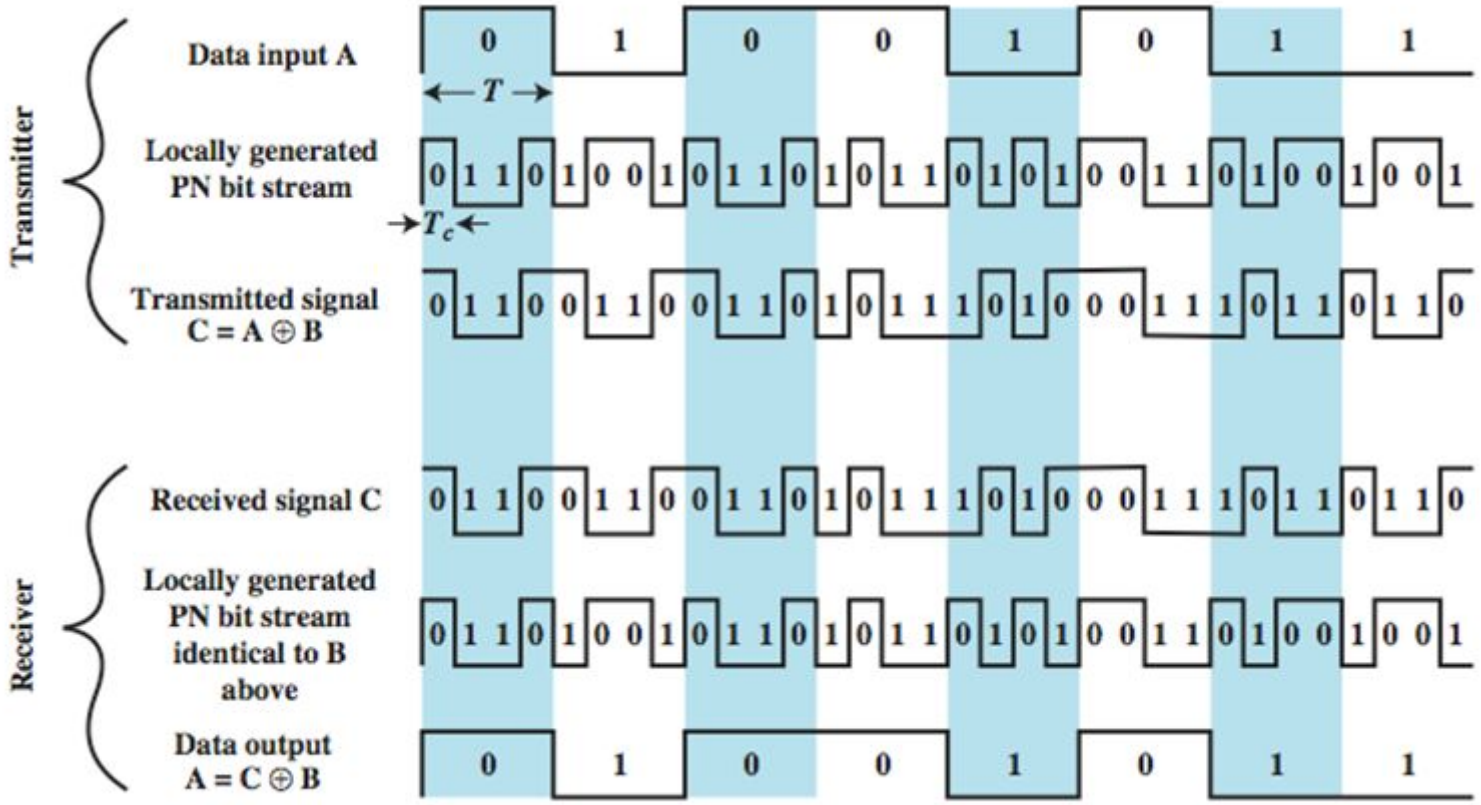
Spread Spectrum Technique

- Data sent using spread spectrum is intentionally **spread over a wide frequency range**
  - Resistant to noise and interference thus increasing the probability that the signal will be received correctly
  - Unlikely to interfere with other signals even if they are transmitted on the same frequency
- Types of Spread Spectrum common in ISM bands:
  - Direct Sequence Spread Spectrum (DSSS)
  - Frequency Hopping Spread Spectrum (FHSS)
  - Orthogonal Frequency Division Multiplexing (OFDM)
  - Chirp Spread Spectrum (CSS)

Model of a Spread Spectrum system

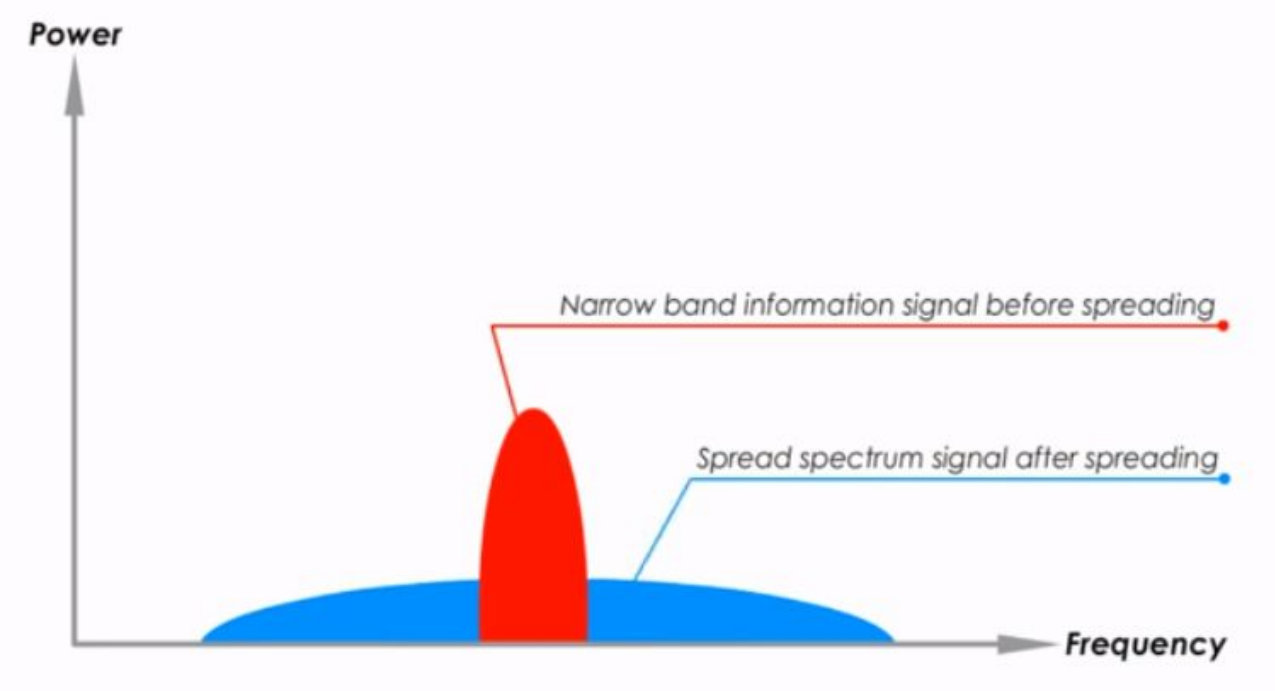


Direct Sequence Spread Spectrum

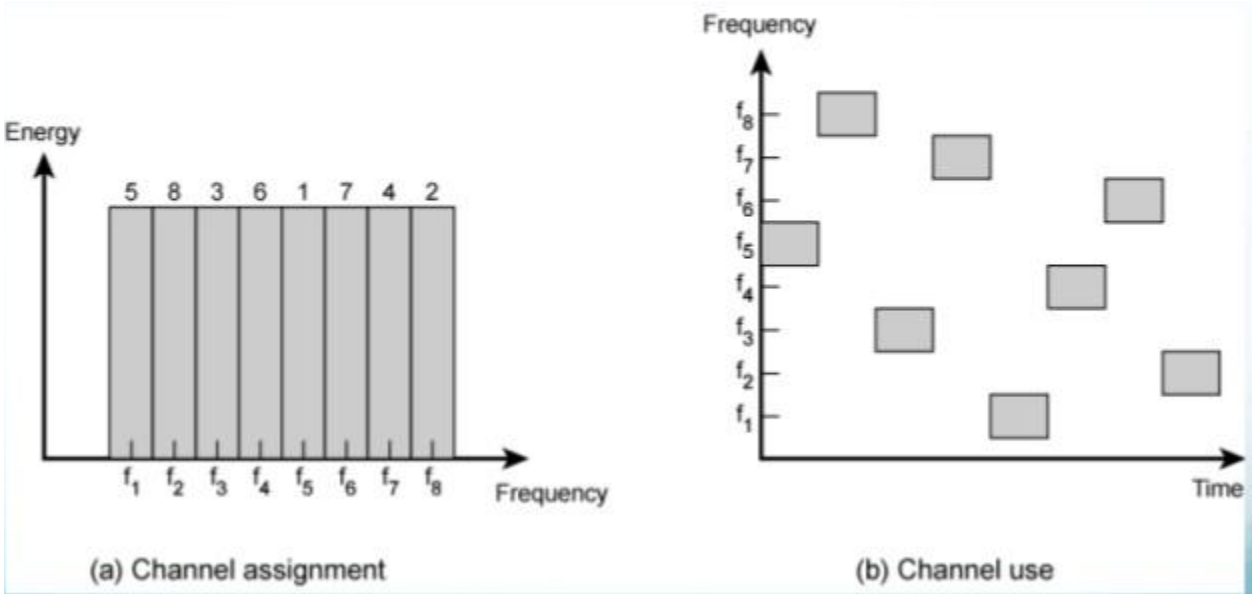


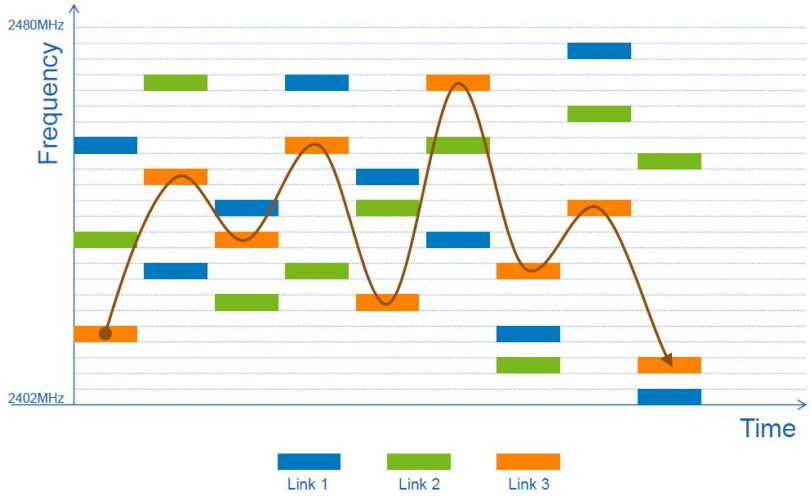
Direct Sequence Spread Spectrum

- Advantages of DSSS:
  - More bandwidth
  - Data are encoded
  - Low power density, noise-like signal

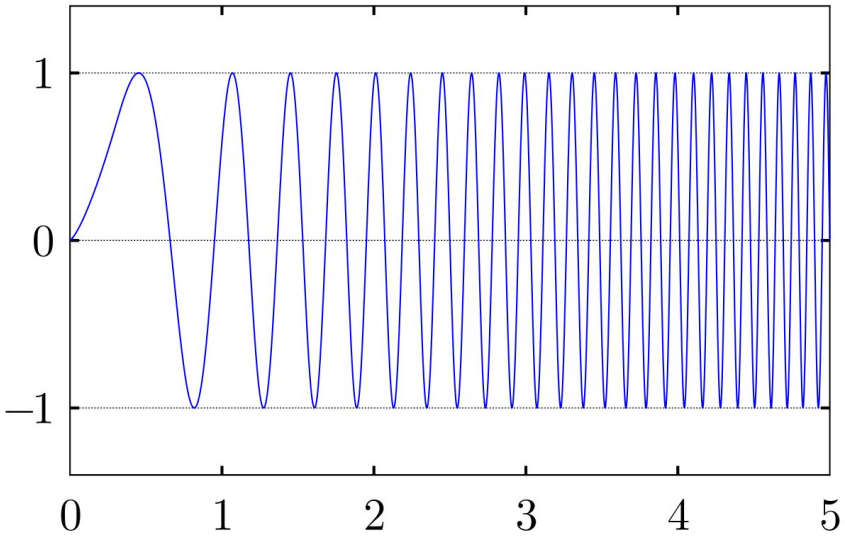


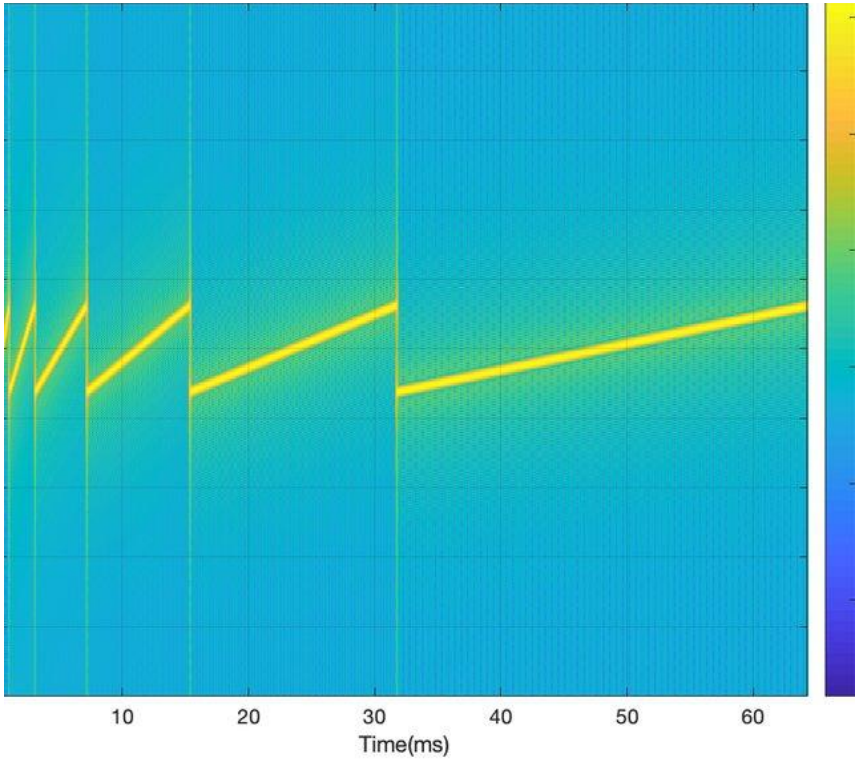
Frequency Hopping Spread Spectrum





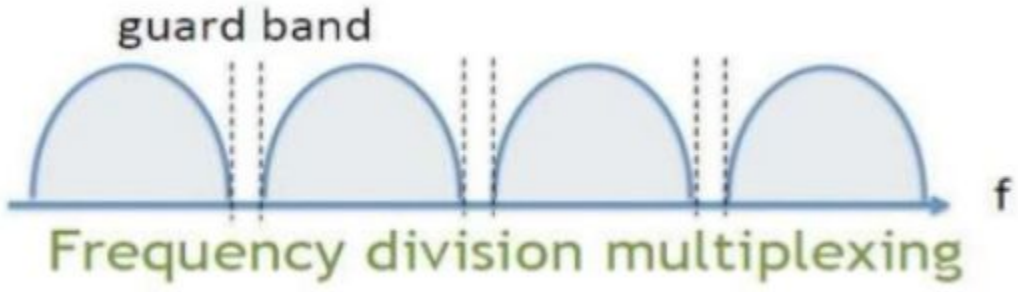
Chirp Spread Spectrum

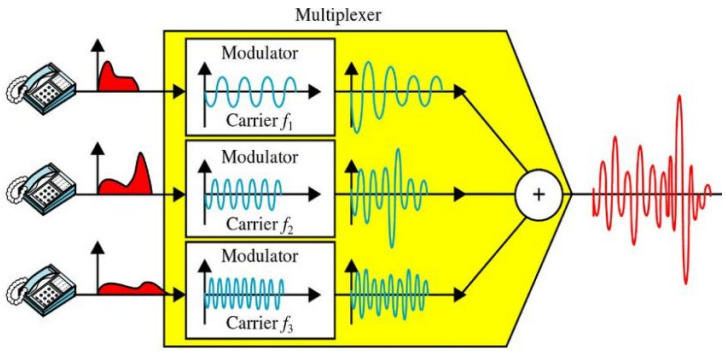


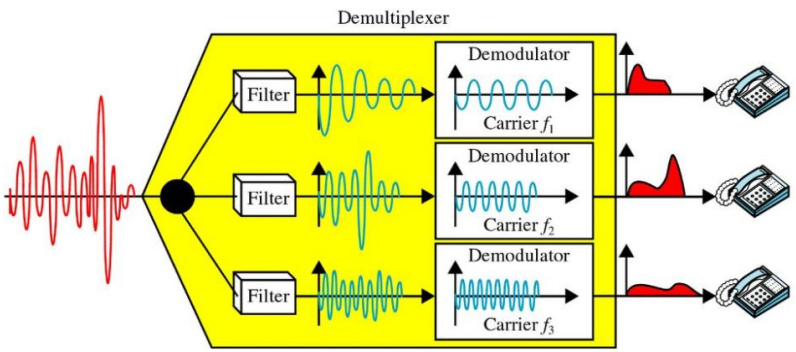


Orthogonal Frequency Division Multiplexing (OFDM)

- Multiplexing is a technique that allows the simultaneous transmission of multiple signals.
- FDM:

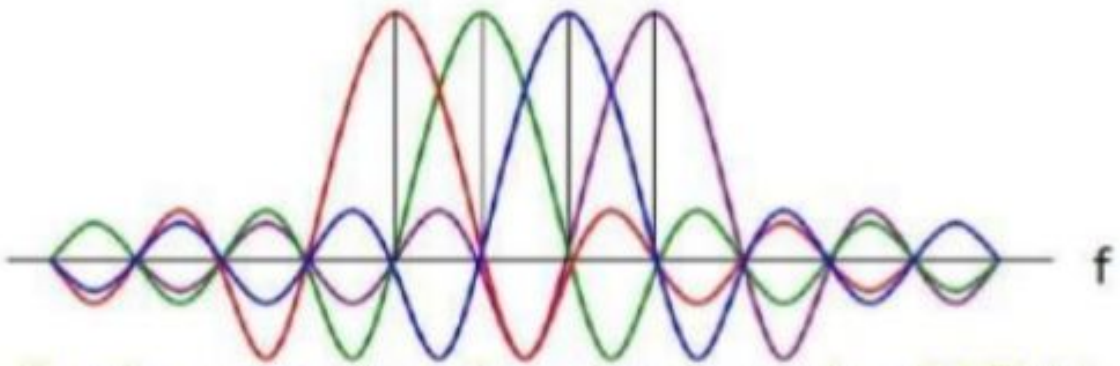






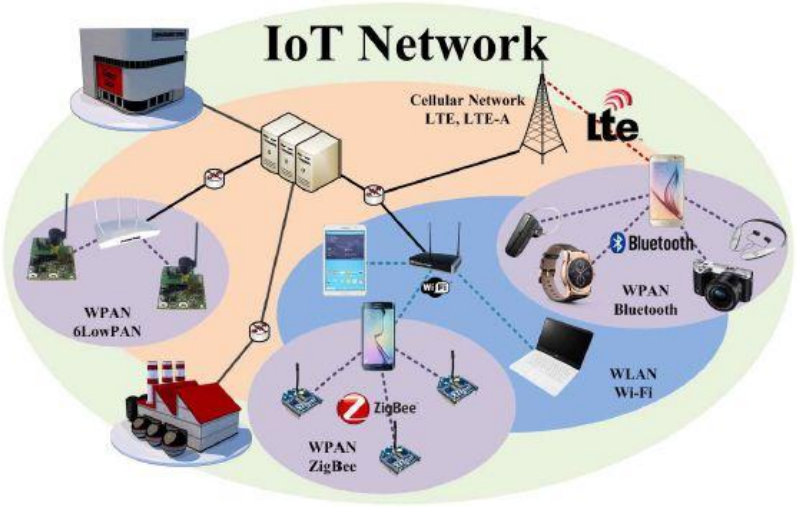
Orthogonal Frequency Division Multiplexing (OFDM)

- OFDM



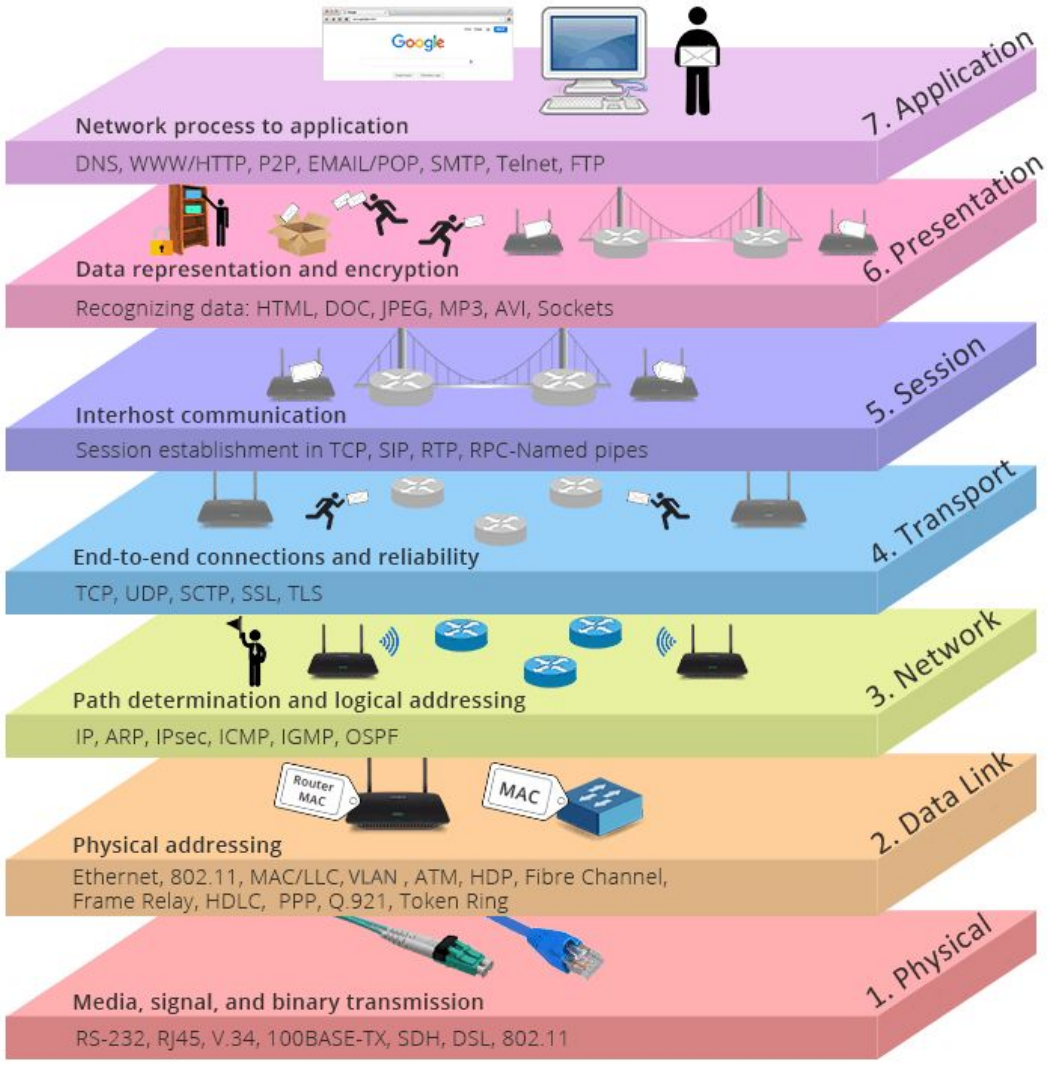
IoT Network

- IoT **not** a single device network
- IoT is a network of internet connected devices that communicate using a common set of **protocols**
- **Network protocols** are a **set of rules** outlining how connected devices communicate across a network to exchange information easily and safely

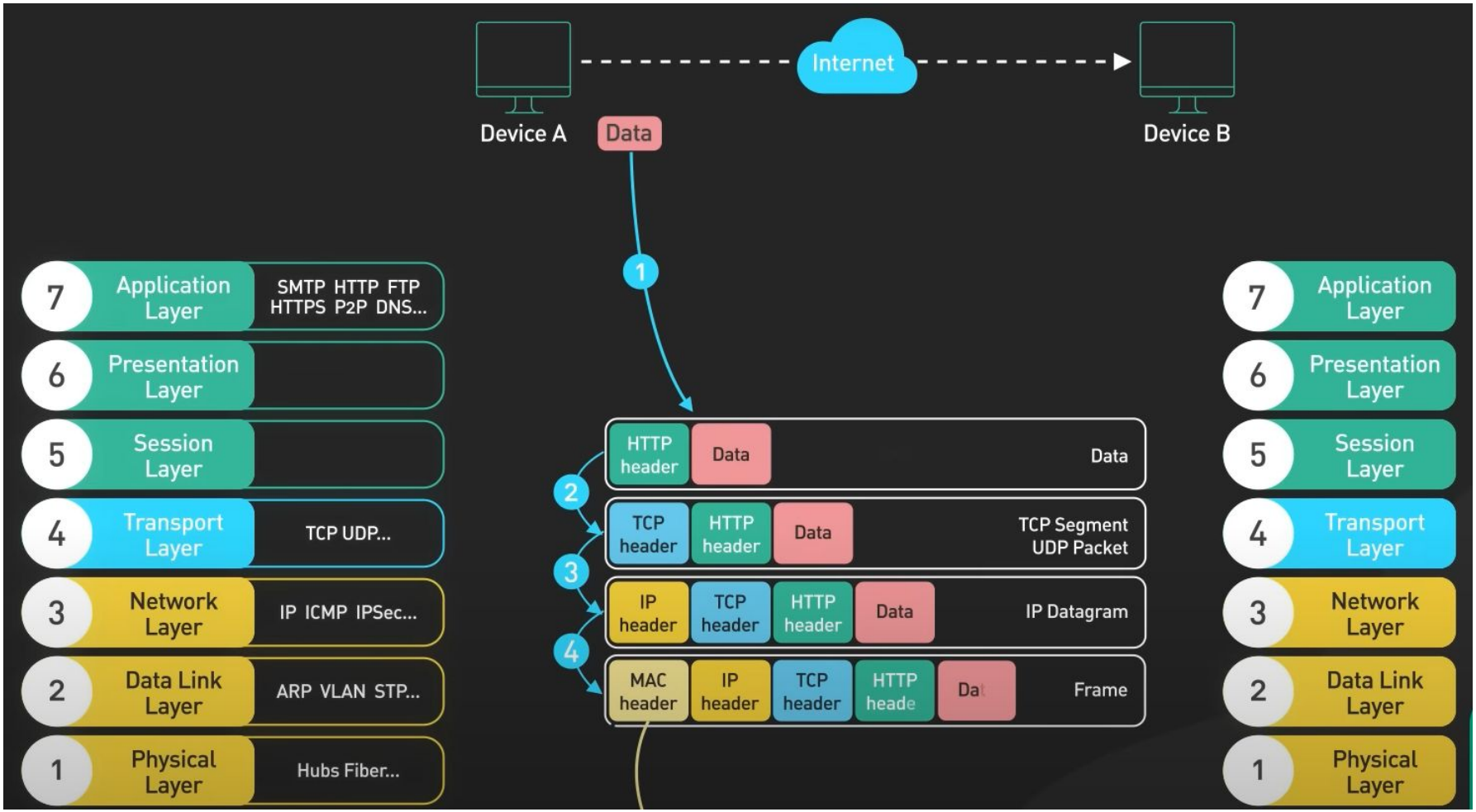


Network Model

- Protocol War (1970s – 1990s)
- Open System Intercommunication (OSI) Model – 1980s
- TCP/IP network (1990s)
- BLE, LPWAN, Cellular, Mesh (2010s)



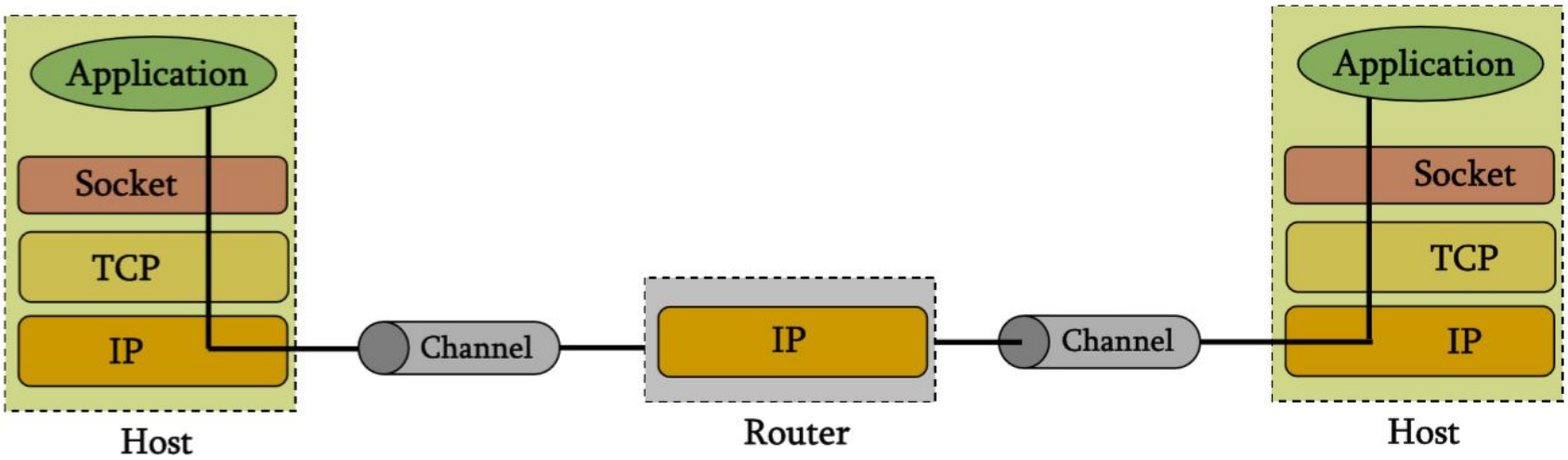
Network Model



Source: https://www.youtube.com/watch?v=0y6FtKsg6J4

Berkeley Socket

- Universally known as **Sockets**
  - is an application programming interface (API)
  - application send and receive data through
- Standard API for networking



Berkeley Socket

- Uniquely identified by
  - Address (MAC, Internet)
  - End-to-end protocol (RFCOMM/TCP or L2CAP/UDP)
  - Port number
- Two types of sockets :
  - Stream sockets: (e.g. RFCOMM) provide reliable byte-stream service
  - Datagram sockets (e.g. L2CAP) provide flexible datagram service

Socket programming

Client-Server communication

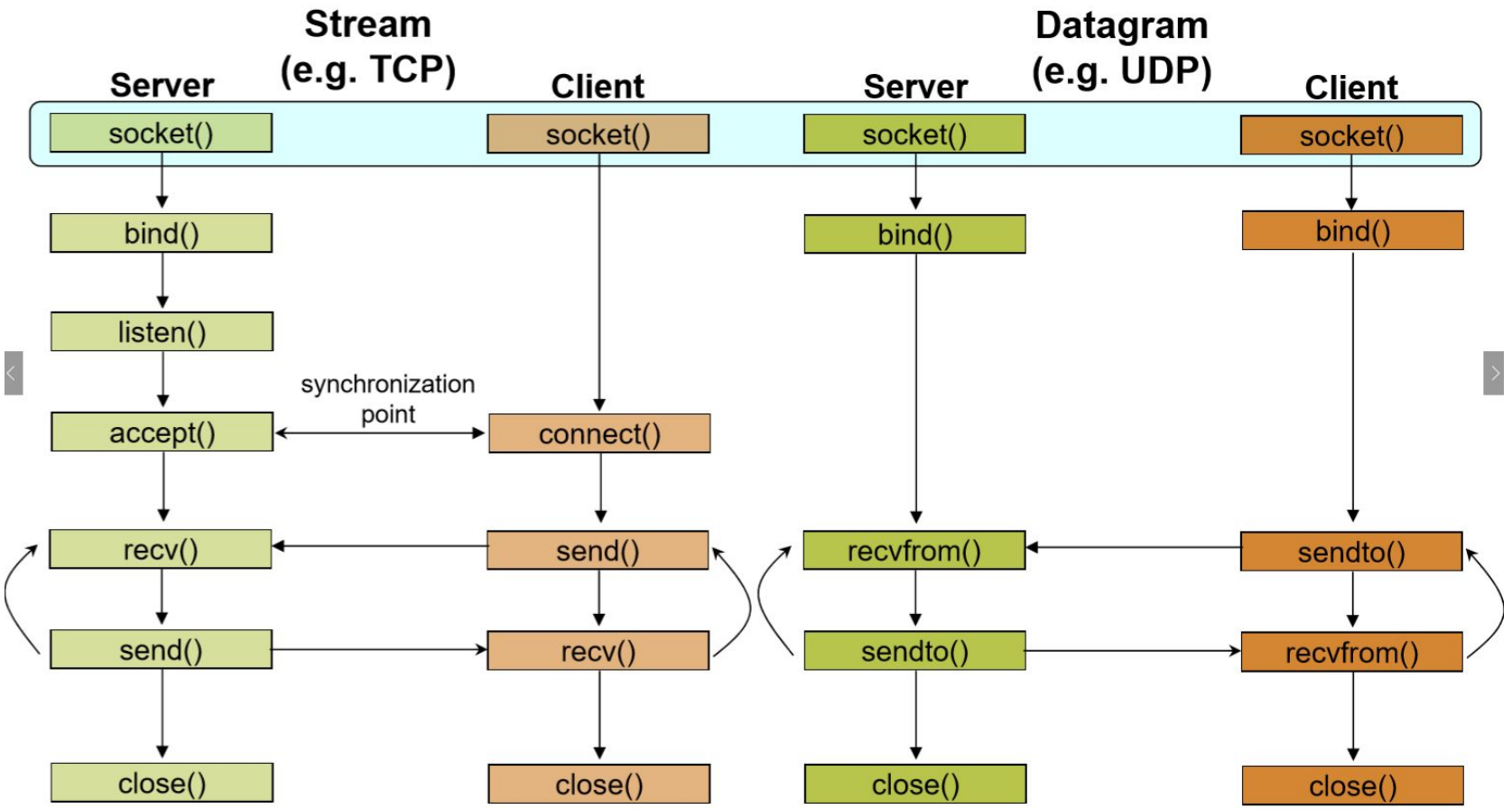
- **Server:**
  - passively waits for and responds to clients
  - Passive socket

- **Client**
  - initiates the communication
  - must know the address and the port of the server
  - Active socket

Sockets Procedures

Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

Client -Server Communication



Socket creation

```
int sockid = socket(family, type, protocol);
```

- **sockid**: socket descriptor, an integer (like a file-handle)
- **family**: integer, communication domain, e.g.,
  - AF_BLUETOOTH, PF_INET, IPv4 protocols, Internet addresses
- type: communication type
  - SOCK_STREAM -reliable, 2-way, connection-based service
  - SOCK_DGRAM -unreliable, connectionless, messages of maximum length
- **protocol**: specifies protocol
  - BTPROTO_L2CAP, BTPROTO_RFCOMM
- socket just creates the interface, does not specify where data will be coming from, nor where it will be going to

Assign address to socket

```
int status = bind(sockid, &addrport, size);
```

- **sockid**: integer, socket descriptor
- **addrport**: struct sockaddr, the (IP) address and port of the machine
- **size**: the size (in bytes) of the addrport structure
- status: upon failure -1 is returned

Example:

```
loc_addr.rc_family = AF_BLUETOOTH;
loc_addr.rc_bdaddr = *BDADDR_ANY;
loc_addr.rc_channel = (uint8_t) 1;
bind(s, (struct sockaddr *)&loc_addr, sizeof(loc_addr));
```

Listen to a socket

```
int status = listen(sockid, queueLimit);
```

- **queuelen**: integer, # of active participants that can "wait"for a connection
- **status**: 0 if listening, -1 if error

- The listening socket is
  - never used for sending and receiving
  - used by the server only as a way to get new sockets

Establish Connection

```
int status = connect(sockid, &foreignAddr, addrlen);
```

- **foreignAddr**: struct sockaddr: address of the passive participant
- **addrlen**: integer, sizeof(name)
- **status**: 0 if successful connect, -1 otherwise

Initialize Connection

```
int s = accept(sockid, &clientAddr, &addrLen);
```

- **s**: integer, the new socket (used for data-transfer)
- **clientAddr**: struct sockaddr, address of the active participant
- **addrLen**: sizeof(clientAddr): value/result parameter
  - must be set appropriately before call
  - adjusted upon return

- Accepted connection is blocking, waits for connection before returning